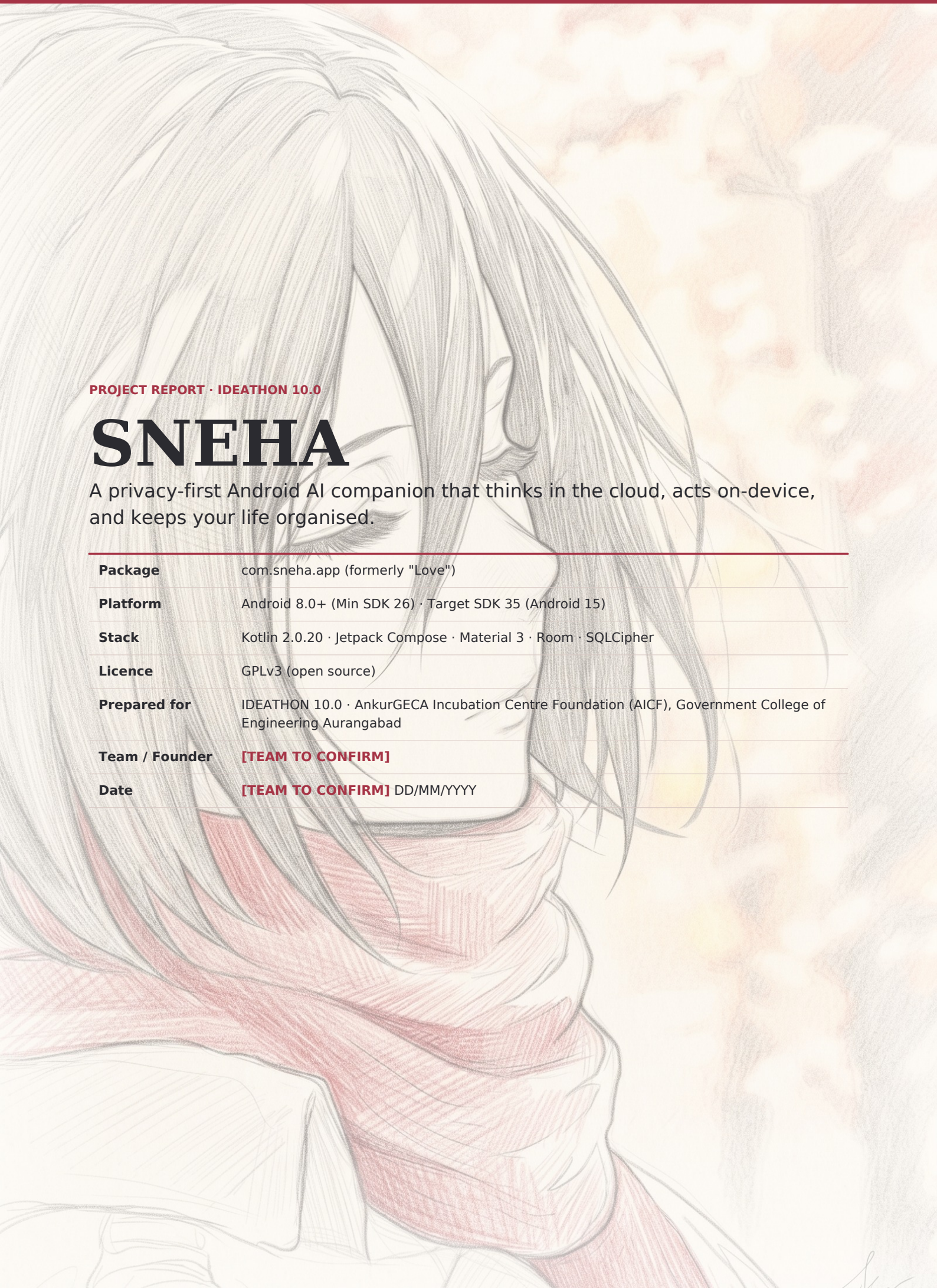


PROJECT REPORT · IDEATHON 10.0

SNEHA

A privacy-first Android AI companion that thinks in the cloud, acts on-device, and keeps your life organised.

Package	com.sneha.app (formerly "Love")
Platform	Android 8.0+ (Min SDK 26) · Target SDK 35 (Android 15)
Stack	Kotlin 2.0.20 · Jetpack Compose · Material 3 · Room · SQLCipher
Licence	GPLv3 (open source)
Prepared for	IDEATHON 10.0 · AnkurGECA Incubation Centre Foundation (AICF), Government College of Engineering Aurangabad
Team / Founder	**[TEAM TO CONFIRM]**
Date	**[TEAM TO CONFIRM]** DD/MM/YYYY

ABOUT THIS DOCUMENT

How this report maps to the AICF pitch deck

The AICF template is a 15-slide pitch deck: title, team name, then twelve content slides covering problem, solution, market, product, business, competition, go-to-market, team, progress, roadmap and the support you want from AICF, closed by a Thank You / Q&A slide. This report is organised in the same order so every slide can be filled straight from a section here. The first half (Sections 1-14) is deck-aligned; the second half (Sections 15-24) is the full technical record of the project, so judges can be given depth on request.

What the template asks for, and where it is answered

Slide	Template heading	What AICF expects	Section
1-2	Title / Startup name	Team or startup name, date (DD/MM/YYYY), presented to IDEATHON 10.0	Cover
3	Problem Statement	Specific problem · who is affected · why it is worth solving	3
4	Solution Overview	Product or service · how it solves the problem · unique selling proposition	4
5	Market Opportunity	Customer segment · target customer · target market · TAM / SAM / SOM · market trends	5
6	Product Details / Prototype	Description · concept visuals or screenshots · stage: Idea, Prototype, MVP or Revenue	6
7	Business Model	Revenue streams · pricing strategy · monetisation approach	7
8	Business Plan	Free-form slide (no prompts given) - costs, milestones, resources	8
9	Competitive Analysis	Key competitors · how you are different · competitive advantage	9
10	Go-To-Market Strategy	Customer acquisition · initial launch plan · sales and marketing channels	10
11	Founding Team	Founder bios and experience · roles · mentors / advisors	11
12	Current Progress	Achievements · early feedback or pilots · recognitions or grants	12
13	Roadmap / Future Plan	Milestones for next 6-12 months · long-term vision · growth plan	13
14	Support Required from AICF	Mentorship · infrastructure / space · funding / grants · networking and exposure	14
15	Thank You / Q&A	Contact name, email / phone, optional QR or social link	14 + 24

How to read the colour tags

Everything marked **[TEAM TO CONFIRM]** is information that only the team can supply (names, dates, traction, pricing decisions) and was not in the project data. Items marked **[PROPOSED]** are reasoned suggestions derived from the project data, written so the team can accept, edit or discard them. Unmarked text is taken directly from the project documentation. No market figures, user counts or financials have been invented.

SECTION 2 · TEMPLATE SLIDE: Title + overview

Executive summary

SNEHA is an Android AI companion built around one idea: a personal assistant should be **warm, useful and private at the same time**. It combines a multi-provider cloud AI failover cascade (ten providers, most with free tiers) with a deterministic on-device layer that handles common commands instantly and offline, and wraps both inside a full personal-productivity suite: tasks, calendar, planning board, habits, journal, focus timer, gym log, alarms, PDF viewer and a Bluetooth-mesh nearby chat. A hands-free voice assistant with a custom wake word ("Sneha") ties it together.

Every sensitive element - API keys, the local database, untrusted content flowing into the AI - is protected by a hardened design: hardware-backed AES-256 key storage, optional SQLCipher database encryption, a prompt-injection guard and a private mode that cuts off all non-private providers.

At a glance

Dimension	Detail
Product type	Native Android app (Kotlin, Jetpack Compose, Material 3), MVVM + unidirectional data flow
AI layer	10-provider failover cascade with circuit breaker; Gemini, Groq, OpenRouter, Cerebras, Together, NVIDIA, Hugging Face, SambaNova, GitHub Models, Ollama (local)
Local intelligence	Deterministic pattern layer with confidence-gated, reversible execution; BM25 memory retrieval; offline template engine
Capabilities	9 capability executors, ~25 tool actions (tasks, calendar, board, memory, settings, navigation, media, date/time, gym)
Data	Room database v16 · 22 entities · 16 migrations · FTS4 global search across 8 sources
Security	AndroidKeyStore AES-256 GCM · PBKDF2 600k iterations · SQLCipher 4.5.3 · prompt-injection guard
Voice	Foreground service, offline wake-word (sherpa-onnx), Android STT/TTS, multilingual wake phrases incl. Hindi
Connectivity	Opt-in Bluetooth mesh chat (BitChat, Noise protocol), MCP client for external tools
Modules	:app · :bitchat · :bitchat-noise
Licence	GPLv3

One-line pitch
SNEHA is the AI companion that never goes down, never has to leak your data, and actually does things for you - on a phone you already own, at near-zero infrastructure cost.

SECTION 3 · TEMPLATE SLIDE: Slide 3 - Problem Statement

Problem statement

What specific problem are you solving?

People who want an AI assistant on their phone face a three-way compromise:

- **Fragility.** Cloud AI apps depend on a single provider. One outage, rate limit or quota exhaustion and the assistant stops working - exactly when it is needed.
- **Privacy.** Mainstream assistants send personal conversations, tasks, journals and habits to remote servers by default, with little control for the user.
- **Fragmentation.** A companion that talks, a task manager, a calendar, a habit tracker, a journal, a workout log and an alarm are normally six different apps that do not share context, so the AI cannot act on the user's real life.

Who is affected?

- Students and young professionals who juggle study, tasks, fitness and routines on one phone.
- Privacy-conscious users who are uncomfortable with cloud-only assistants.
- Users in India and other multilingual markets whose language (Hindi, Hinglish) is poorly served: the project even budgets tokens per script (Devanagari counts roughly 2.2 characters per token vs 4.0 for English).
- Users with limited budgets who cannot pay for premium AI subscriptions.

Why is it worth solving?

- Free-tier AI is abundant but unreliable on its own; combining many of them with health tracking turns scattered free quotas into one dependable service.
- Most "AI assistants" only chat. Users need an assistant that executes: create the task, schedule the event, log the set, set the alarm.
- Prompt injection, key theft and data leakage are growing risks as assistants gain tools. Security has to be designed in, not patched on.
- Emotional wellbeing is part of productivity: a companion that remembers you, checks in morning and evening, and offers a private journal addresses more than the to-do list.

Suggested deck sentence [PROPOSED]
"Today's AI assistants are fragile, public and disconnected from your real life. SNEHA is resilient, private and wired into your day."

SECTION 4 · TEMPLATE SLIDE: Slide 4 - Solution Overview

Solution overview

What is the product?

SNEHA is a native Android application that behaves as a personal companion and a doer. The user can type or speak to it; it answers in a warm, concise voice and can take actions in the user's own data via a typed, validated tool system.

How does it solve the problem?

Problem	SNEHA's answer
Single point of failure	CascadeAiService walks a ranked list of 10 providers. A CircuitBreaker benches a provider after 2 consecutive failures (60 s cooldown, 15 min on rate limit) and the request transparently fails over to the next. Per-provider health, latency and success rate are exposed in Settings.
Latency and offline use	DeterministicLayer resolves quick-add tasks, calendar and board commands locally (confidence 0.85-0.95). ConfidenceRouter only auto-executes at confidence ≥ 0.9 and only if reversible, with undo.
Privacy	Private mode skips every non-private provider (local Ollama only). Keys live in hardware-backed AndroidKeyStore; the DB can be SQLCipher-encrypted; no plaintext fallback exists.
Prompt-injection and tool abuse	All untrusted text (tool output, MCP, PDFs, web, nearby chat) is wrapped in delimiters with a SHA-256 hash. Destructive tools need approval; JSON Schema 2020-12 validation with bounded self-repair (max 2 retries).
Fragmentation	Single app: chat, tasks, calendar, board, habits, journal, focus, gym, alarms, PDF, media, nearby chat, with FTS4 global search across everything.
Hands-free use	Foreground voice service with an offline wake word, STT, AI/tool loop and TTS reply.

Unique selling proposition

- **Resilient by design:** 10-provider cascade with circuit breaker; also works with no cloud at all (deterministic layer + local template engine).
- **Private by design:** hardware-backed secrets, optional encrypted DB, private mode, injection guard, GPLv3 transparency.
- **Agentic, safe execution:** risk-tiered capabilities (READ / WRITE / DESTRUCTIVE), approval gates, checkpoints and undo.
- **Multilingual and voice-native:** Hindi/Hinglish-aware token budgeting and wake phrases, offline wake word.
- **All-in-one:** companion plus productivity suite that share one memory.
- **Near-zero cost to run:** architecture leans on free-tier providers and on-device processing.

SECTION 5 · TEMPLATE SLIDE: Slide 5 - Market Opportunity

Market opportunity

> **Note on figures**
> The project data contains no market-size numbers, and none have been invented. TAM / SAM / SOM below is a structure to be filled with cited figures from a source of the team's choice **[TEAM TO CONFIRM]**.

Customer segments [PROPOSED]

Segment	Why SNEHA fits	Features that matter
Students (college, competitive exams)	Study mode, focus timer, PDF viewer, tasks and habits in one place; budget-constrained, free-tier friendly	Study mode, focus, PDF, tasks, habits, memory
Young professionals	Calendar + tasks + voice assistant + morning briefing	Quick-add, calendar, briefing, alarms linked to events
Fitness users	Gym log with Epley 1RM PR detection and progression recommendations	Gym, alarms linked to sessions
Privacy-conscious / open-source users	GPLv3, private mode, encrypted storage, local Ollama support	Private mode, SQLCipher, security model
Hindi / multilingual users	Devanagari-aware token budgeting and wake phrases	Voice, multilingual wake words

Target customer (initial beachhead) [PROPOSED]

Android-using students and early-career professionals in India, aged roughly 18-30, who are already comfortable with AI chat tools, and who value free or low-cost, private and voice-friendly assistants.

Market size framework (TAM / SAM / SOM)

Layer	Definition for SNEHA	Figure
TAM	All smartphone users worldwide who use or would use an AI assistant	**[TEAM TO CONFIRM]**
SAM	Android users in India (and similar multilingual markets) who use AI assistants or productivity apps	**[TEAM TO CONFIRM]**
SOM	Realistic 12-24 month share: students / early professionals reached through campus and open-source channels	**[TEAM TO CONFIRM]**

Market trends (qualitative)

- Rapid growth of free and open model APIs (Gemini, Groq, Cerebras, OpenRouter, GitHub Models and others offer free tiers) lowers the cost of building AI apps - the cascade is built on exactly this.
- Shift from chat to agents: assistants are expected to call tools and take actions.
- Rising concern over privacy, prompt-injection and data sovereignty; on-device and private modes are becoming a differentiator.
- Voice-first interaction and Indic-language support are expanding in the Indian market.
- Open-source (GPLv3) software gaining trust in education and government ecosystems.

SECTION 6 · TEMPLATE SLIDE: Slide 6 - Product Details / Prototype

Product details and prototype

Description of the product

SNEHA is a single-activity Compose application (singleTask launch mode) with a glassmorphic UI, eight colour palettes, Material You dynamic colour and a shared looping video background decoded once behind the NavHost. Everything is built lazily on launch, and check-in alarms are re-registered automatically.

Feature map

Area	What the user gets
Chat	Streaming AI chat with tool-call visualisation and provider metadata; modes: Companion, Planner, Executor, Study
Home	Today dashboard: due tasks, today's events, next alarm, today's workout; global search
Tasks	Natural-language quick-add (title, due date, priority, tags, recurrence, project), projects, subtasks, status
Calendar	Monday-first 42-cell month grid, events, recurrence, overlap detection, free-slot finder
Board	Interactive canvas of sticky notes, Kanban columns, WIP limits, templates, checklists, six-colour palette
Memory vault	Auto-extracted personal facts, search, pin, edit, delete, sensitivity flags, importance levels
Habits, journal, focus	Habit streaks; private mood journal; work/break focus timer; privacy centre with data wipe
Gym	Exercise library, sessions, set logging, 1RM PR detection (Epley), progression engine
Alarms	Exact alarms, full-screen ringing UI, snooze, missions, Quick Settings tile, home widget, link to calendar events or gym sessions
Voice	Wake word "Sneha" (and variants), spoken replies, proactive morning/evening check-ins
Media and PDF	In-app Spotify/YouTube player with background playback; PDF viewer and library with share-sheet import
Nearby chat	Opt-in Bluetooth mesh rooms and Noise-encrypted direct messages
Settings	Provider keys, health probes, cascade diagnostics, theme, privacy toggles, custom OpenAI-compatible endpoint

Concept visuals / screenshots

> **Add before submission [TEAM TO CONFIRM]**
> Suggested shots: Home · Chat with a tool call · Board · Cascade diagnostics · Voice · Alarm · Nearby Chat.

Development stage

Recommended declaration: advanced prototype / pre-MVP [PROPOSED]. The app is feature-rich and builds on Android 8-15, but the documentation lists known open items: PDF text extraction returns empty (PdfBox not yet integrated, "Phase 0"), a legacy non-spec MCP client is still being replaced, Hilt is being introduced gradually, and nearby-chat peer identities are not independently verified. No revenue is recorded in the project data.

SECTION 7 · TEMPLATE SLIDE: Slide 7 - Business Model

Business model

Important context
The project data does not define pricing or revenue. SNEHA is licensed GPLv3, which allows commercial use but requires derivative works to remain open source. The options below are therefore built around services, convenience and support rather than closed-source licence sales. They are proposals for the team to choose from.

Revenue stream options [PROPOSED]

Stream	How it works	Fit
SNEHA Pro subscription	Optional paid tier: managed provider cascade (no need to bring your own keys), cloud backup/sync, higher limits	Strong - removes the main setup friction for non-technical users
Bring-your-own-key (free)	Core app free; users add free-tier keys for the ten supported providers	Keeps cost near zero and fits GPLv3 / privacy story
Institutional / campus licences	Hosted deployments or support for colleges and coaching institutes (study mode, PDF, focus, tasks)	Good - aligned with AICF / GECA ecosystem
Support and customisation	Paid integration, custom capabilities and MCP connectors for organisations	Natural for open-source projects
Sponsorship / grants	Open-source grants, donations, incubator funding	Early-stage bridge

Pricing strategy [PROPOSED]

- Freemium: full local productivity suite and bring-your-own-key AI free forever.
- Pro tier priced at a student-friendly monthly price point **[TEAM TO CONFIRM]**.
- Institutional pricing per seat or per department **[TEAM TO CONFIRM]**.

Monetisation approach

Grow on trust first (open source, privacy, zero lock-in), then monetise convenience: managed AI access, sync and support. Because infrastructure leans on free-tier providers and on-device processing, marginal cost per free user is very low, which keeps the freemium model sustainable.

SECTION 8 · TEMPLATE SLIDE: Slide 8 - Business Plan (free-form)

Business plan

The template gives no prompts for this slide, so a compact plan covering cost, resources and milestones is recommended.

Cost structure (what the architecture implies)

- **Cloud AI:** near zero for BYOK users because the cascade targets free tiers; managed Pro tier would add metered API cost **[TEAM TO CONFIRM]**.
- **Infrastructure:** no mandatory backend today - data lives on-device. Any sync / managed-key service would be a new cost line.
- **Distribution:** Play Store developer account, optional F-Droid release (GPLv3-compatible).
- **People:** Android/Kotlin development, security review, design, community support **[TEAM TO CONFIRM]**.

Key resources

- Codebase with 3 modules, 22 entities, ~9 capability executors, 10 provider integrations and extensive documentation (ARCHITECTURE, AUDIT, THREAT_MODEL, STUDY_NOTES).
- Open-source libraries: BouncyCastle, Tink, Nordic BLE, sherpa-onnx, Room, SQLCipher, Media3, OkHttp, WorkManager.
- Design assets: eight themes, glass components, splash and background video.

12-month plan on a page [PROPOSED]

Quarter	Goal	Output
Q1	Stabilise and publish	Closed beta, crash/analytics-free feedback loop, screenshots and demo video, Play Store listing
Q2	Close technical gaps	PdfBox text extraction, spec-compliant MCP client only, peer verification in nearby chat, security review
Q3	Campus pilots	Pilots with colleges / coaching groups, study-mode refinements, collect testimonials
Q4	Monetise convenience	Launch optional Pro (managed cascade + sync) and institutional offering

Funding need and use of funds

Amount requested and allocation **[TEAM TO CONFIRM]**. Suggested categories: development hours, security audit, devices for testing, cloud credits for Pro tier pilot, design and marketing assets.

SECTION 9 · TEMPLATE SLIDE: Slide 9 - Competitive Analysis

Competitive analysis

Competitors are best understood by category. Product names are illustrative; the team should verify current features before presenting **[TEAM TO CONFIRM]**.

Key competitors

Category	Examples	Where they are strong	Where SNEHA differs
General AI assistants	Google Gemini app, ChatGPT, Microsoft Copilot	Powerful single model, brand, polish	Not tied to one provider; private mode; data stays local; deeper action on a personal productivity suite
Voice assistants	Google Assistant, Alexa, Siri	System integration, mature wake word	LLM reasoning plus offline wake word, memory and open source; multilingual wake phrases
AI companion apps	Replika, Character-style companion apps	Emotional engagement	Companion plus real tools: tasks, calendar, habits, gym; privacy-first
Productivity apps	Todoist, Notion, Google Calendar, Habitica, Strong-style gym loggers	Depth in one domain	One memory across all domains with an AI that can act in each
Private / mesh chat	Bitchat-style Bluetooth messengers	Offline mesh messaging	Mesh chat is one feature inside a bigger assistant

How are you different?

- Failover cascade with circuit breaker: reliability others do not offer because they rely on one backend.
- Deterministic-first routing: instant, offline execution of common commands with undo.
- Security as architecture: hardware-backed keys, no plaintext fallback, SQLCipher, injection guard, deny-wins permissions.
- Agent modes (Instant, Smart, AI Only, Orchestrator, Plan, Explore) with step budgets and checkpoints.
- Open source under GPLv3 with a published threat model.

Competitive advantage and moat

The moat is the integration depth and the trust model, not a single model: a user whose tasks, memory, habits and routines already live in SNEHA, protected on their own device, has little reason to move. Provider-agnostic architecture also means SNEHA improves automatically whenever any provider improves or a new free tier appears.

SECTION 10 · TEMPLATE SLIDE: Slide 10 - Go-To-Market Strategy

Go-to-market strategy

Customer acquisition approach [PROPOSED]

- **Campus-first:** start with GECA and neighbouring colleges; students are the highest-fit segment and spread by word of mouth.
- **Open-source community:** GitHub, F-Droid, Reddit r/androiddev and r/selfhosted-style channels; developers become advocates and contributors.
- **Content:** short demo videos (wake word, failover in action, tool calls with undo), "how I built a 10-provider cascade" technical write-ups.
- **Creators and student ambassadors** for study-mode and productivity use cases.

Initial launch plan

Phase	Action
1. Closed beta (4-6 weeks)	Invite 50-100 campus users, gather qualitative feedback, fix stability and onboarding
2. Public beta	Release on Play Store open testing and F-Droid / GitHub releases
3. Launch	Demo video, launch post, ambassador programme, campus demos

Sales and marketing channels

- Google Play and F-Droid listings with screenshots and a short video
- GitHub repository and README as the primary landing page
- College events, hackathons and AICF network showcases
- Social: Instagram Reels / YouTube Shorts / LinkedIn for founder story
- Institutional outreach to colleges and coaching centres for the paid offering

Onboarding as a growth lever

The existing onboarding flow (name, concerns, check-in times, enable/disable) personalises the app in under a minute, and morning/evening check-ins create a daily habit loop that supports retention.

SECTION 11 · TEMPLATE SLIDE: Slide 11 - Founding Team

Founding team

The project data does not include team information. Use the table below as the slide skeleton.

Name	Role	Relevant experience / skills	Responsibilities
[TEAM TO CONFIRM]	Founder / Lead Android developer	[TEAM TO CONFIRM]	Architecture, AI cascade, security
[TEAM TO CONFIRM]	[TEAM TO CONFIRM]	[TEAM TO CONFIRM]	[TEAM TO CONFIRM]
[TEAM TO CONFIRM]	[TEAM TO CONFIRM]	[TEAM TO CONFIRM]	[TEAM TO CONFIRM]

Mentors and advisors

[TEAM TO CONFIRM] List any faculty guide, industry mentor or open-source maintainer who advises the project.

Skills the project already demonstrates

- Native Android engineering: Kotlin, Compose, Room, WorkManager, foreground services, Media3
- Applied AI engineering: multi-provider routing, function calling, token budgeting, prompt assembly
- Security engineering: KeyStore, PBKDF2, SQLCipher, threat modelling, prompt-injection defence
- Systems integration: BLE mesh, Noise protocol, MCP, offline wake-word (ONNX)
- Product and UX: glassmorphic design system, accessibility scaling, onboarding

SECTION 12 · TEMPLATE SLIDE: Slide 12 - Current Progress

Current progress

Achievements so far (from the codebase)

Metric	Value
Gradle modules	3 (:app, :bitchat, :bitchat-noise)
Room database	Version 16 · 22 entities · 16 migrations · 14 DAOs · FTS4 global search
UI feature modules	21 screen packages, 20 navigation routes
AI providers integrated	10 (9 cloud/free-tier + local Ollama), plus custom OpenAI-compatible endpoint
Capability executors	9 (tasks, calendar, board, memory, settings, navigation, media, datetime, gym)
Agent modes	6 (Instant, Smart, AI Only, Orchestrator, Plan, Explore) + 4 personality modes
Themes	8 palettes · light/dark/system · Material You
Android services / receivers	5 services, 3 receivers, 3 activities, 1 widget, 1 Quick Settings tile
Documentation	ARCHITECTURE, AUDIT, CHANGELOG, THIRD_PARTY, THREAT_MODEL, 4 study notes; 17 reference repos studied

Engineering milestones completed

- Spec-compliant MCP client on the Kotlin SDK (JSON-RPC 2.0, Streamable HTTP with SSE fallback)
- Hardware-backed key storage with refusal-on-failure policy and PBKDF2 fallback
- Opt-in SQLCipher-encrypted database
- Offline wake-word engine with transcript fallback and multilingual phrase parser
- Bluetooth mesh nearby chat integrated from BitChat with Noise protocol
- Gym engine with Epley 1RM and progression recommendations
- Per-script (Devanagari-aware) token estimation and prompt assembly with failover re-assembly

Early feedback, pilots, recognitions or grants

None recorded in the project data. **[TEAM TO CONFIRM]** Add testers, pilot users, hackathon results, awards or grants here; even a handful of quotes from beta users strengthens this slide.

SECTION 13 · TEMPLATE SLIDE: Slide 13 - Roadmap / Future Plan

Roadmap and future plan

The roadmap below is derived from the open items and design intentions recorded in the project documentation **[PROPOSED]**.

Next 6-12 months

Horizon	Milestone	Source in project data
0-3 months	Integrate PdfBox for real PDF text extraction (enables summarise / Q&A on PDFs)	PdfEngine "Phase 0" returns empty text
0-3 months	Retire the legacy MCP client in favour of the spec-compliant SDK client	Legacy client marked "being replaced"
0-3 months	Closed beta, bug-fix pass, onboarding polish	Distribution readiness
3-6 months	Independent peer identity verification in nearby chat; clarify public-room vs encrypted DM in UI	Documented limitation of initial interface
3-6 months	Continue Hilt introduction for new code; expand tests	Design decision 1
3-6 months	Expand deterministic coverage (more local commands, habits, journal, gym)	Deterministic-first principle
6-12 months	Optional Pro tier: managed cascade, encrypted sync / backup	Business model
6-12 months	More MCP connectors and capabilities; Orchestrator subtasks in daily use	Agent modes
6-12 months	Campus pilots and institutional offering	GTM

Long-term vision

A private personal operating layer for your phone: an assistant that knows you, acts across your apps through MCP, works with or without the cloud, and is trusted because it is open, auditable and on-device first.

Product and business growth plan

- Product: more languages and voices, smarter on-device models, richer automations with checkpoints and undo, wearable and tablet support.
- Business: grow community, add Pro and institutional revenue, partner with colleges and incubators, pursue grants for open-source security work.

SECTION 14 · TEMPLATE SLIDE: Slides 14-15 - Support, Thank You / Q&A;

Support required from AICF

Support area	Specific ask (proposed)	Why it helps
Mentorship	Product-market-fit mentor; Android / security advisor; legal mentor for GPLv3 + commercial model	Sharpens pricing, licence strategy and security posture
Infrastructure / space	Incubation desk, test device lab, cloud credits for managed-AI pilot, secure Wi-Fi for BLE mesh testing	Enables multi-device testing of nearby chat and voice
Funding / grants	Seed grant **[TEAM TO CONFIRM]**	Security audit, devices, design and launch assets, Pro tier pilot
Networking and exposure	Introductions to colleges, alumni, industry; demo days; media	Campus pilots and early adopters

Thank You / Q&A slide

Field	Value
Contact person	**[TEAM TO CONFIRM]**
Email / phone	**[TEAM TO CONFIRM]**
Repository / QR (optional)	**[TEAM TO CONFIRM]**

Questions judges may ask, with answers from the project data

Question	Answer
What if a provider goes down?	The cascade benches it after 2 failures and moves to the next; if all cloud tiers fail, deterministic and template engines still work.
Is user data sent to the cloud?	Only the prompt content for the chosen provider. Private mode restricts to local-only (Ollama). DB can be fully encrypted.
How do you stop prompt injection?	Untrusted content is delimited and hashed; destructive actions require approval; tool args are schema-validated.
How do you earn money if it is open source?	Convenience services (managed cascade, sync), institutional support and grants - see Section 7.
What is not done yet?	PDF text extraction, retirement of the legacy MCP client, nearby-chat peer verification - see Section 13.

PART II

Technical record

The sections below preserve the full project documentation in a structured form, so that deck claims can be traced to specifics.

SECTION 15 · TEMPLATE SLIDE: Reference

Architecture and tech stack

Modules

Module	Purpose
:app	Application: Compose UI, data layer, AI agent, voice, PDF, alarms, tasks, calendar, board, habits, journal, focus, gym, nearby chat
:bitchat	Nearby chat library: BouncyCastle, Tink, Nordic BLE, CameraX, ML Kit, OkHttp, WorkManager, Play Services Location, security-crypto
:bitchat-noise	Pure Java Noise protocol crypto implementation

Technology stack

Layer	Technology
Language	Kotlin 2.0.20
UI	Jetpack Compose BOM 2024.09.03, Foundation, Material 3
Architecture	MVVM, unidirectional data flow (StateFlow / SharedFlow), Clean Architecture
Database	Room 2.6.1 with KSP; v16; 22 entities; FTS4 global_search_index
Persistence	DataStore Preferences 1.1.1
Async	Kotlin Coroutines 1.8.1 and Flow
Media	androidx.media3 (ExoPlayer and UI) 1.4.1
Navigation	Navigation Compose 2.8.1 (type-safe routes)
Build	Gradle 8.5.2 / AGP 8.5.2, Version Catalogs, KSP 2.0.20-1.0.25, Parcelize, Room schema export, JitPack for sherpa-onnx AAR
Security	AndroidKeyStore AES-256 GCM, PBKDF2-HMAC-SHA256 (600k iterations), SQLCipher 4.5.3
Networking / work	OkHttp 4.12.0 · WorkManager 2.9.0
MCP	modelcontextprotocol/kotlin-sdk (JSON-RPC 2.0, Streamable HTTP with SSE fallback)
Wake word	sherpa-onnx KeywordSpotter (reflection-based optional dependency)
SDK levels	Min 26 (Android 8.0) · Target 35 (Android 15) · Compile 35

Application entry

SnehaApplication lazily initialises Room, preferences, the cascade, capability router, tool registry, deterministic layer and repositories, migrates legacy keys and re-registers check-in alarms.
MainActivity (singleTask) hosts the NavHost, handles shared YouTube/Spotify links and PDFs, requests RECORD_AUDIO and POST_NOTIFICATIONS and starts the voice service.

SECTION 16 · TEMPLATE SLIDE: Reference

AI and agent core

Provider cascade (FreeProvider)

Provider	Model	Native tools	System prompt	Context
GEMINI	gemini-3.8-flash (native API)	Yes	Yes	1M tokens
GROQ	openai/gpt-oss-20b	Yes	Yes	8K
OPENROUTER	google/gemma-3-27b-it:free	Yes	No (merged)	32K
CEREBRAS	llama-3.3-70b	Yes	Yes	8K
TOGETHER	Llama-3.3-70B-Instruct-Turbo	Yes	Yes	16K
NVIDIA	meta/llama-3.3-70b-instruct	Yes	Yes	16K
HUGGINGFACE	Llama-3.1-8B-Instruct	No (code-block fallback)	Yes	8K
SAMBANOVA	Meta-Llama-3.3-70B-Instruct	Yes	Yes	16K
GITHUB_MODELS	openai/gpt-4o-mini	Yes	Yes	8K
OLLAMA (local)	llama3.2:3b	Yes	Yes	4K

Default order: GROQ, OPENROUTER, CEREBRAS, TOGETHER, NVIDIA, HUGGINGFACE, SAMBANOVA, GITHUB_MODELS, GEMINI, OLLAMA. Users can reorder tiers (tier_order preference) and add a custom OpenAI-compatible endpoint.

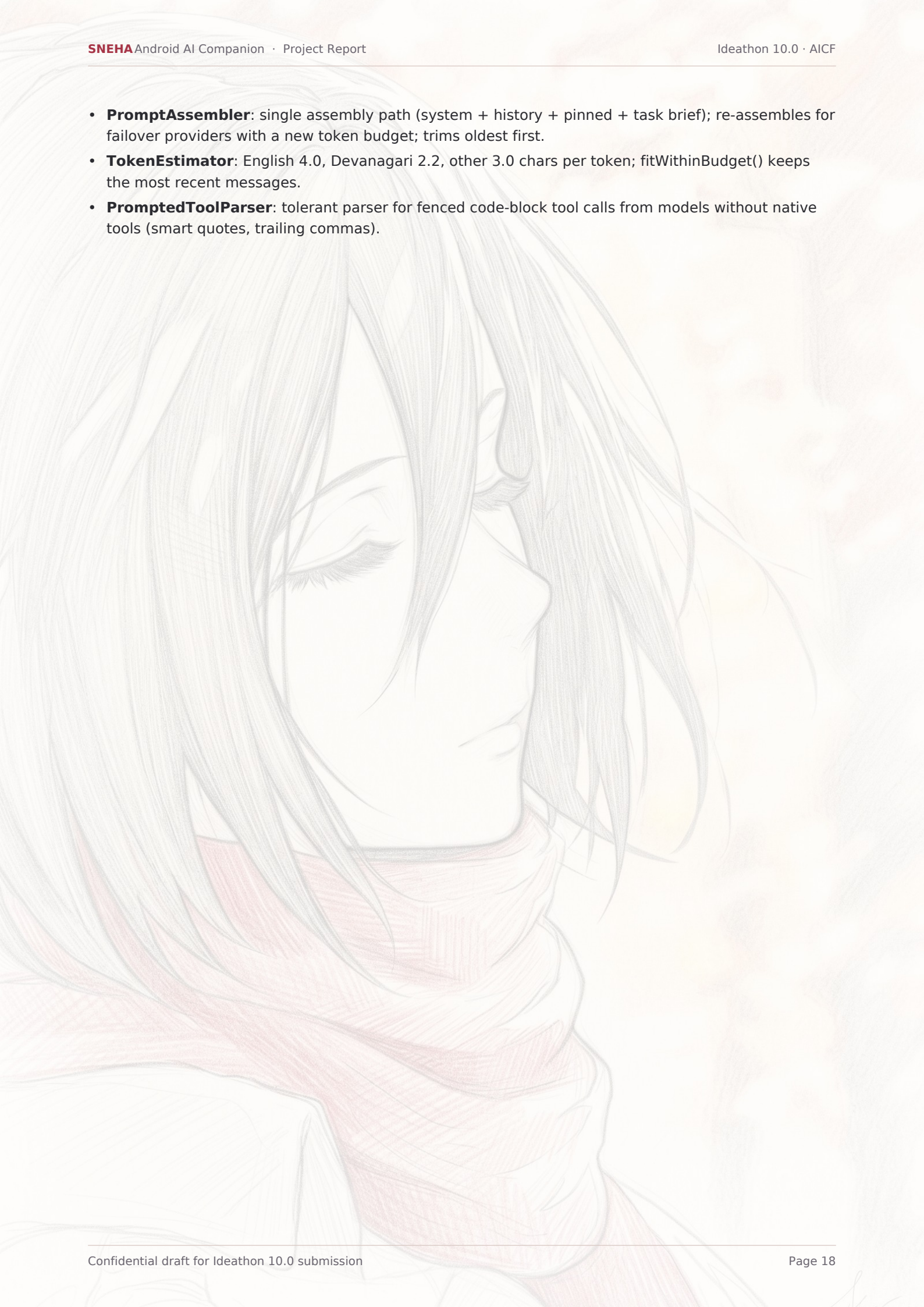
Cascade mechanics

- **AiService** contract: generateReply(userMessage, history, memories, toolSpecs, toolResults) returning Success / Failure / ToolCall; generateReplyStream returning Flow<String>.
- **CascadeAiService** iterates ProviderTiers via pickTier(), records outcomes in a CircuitBreaker, benches failing providers, skips non-private tiers in private mode, and returns CascadeDiagnostics (health, success rate, average latency, bench duration, last error). probeAll() runs health checks.
- **TierHealth**: failure threshold 2, cooldown 60 s, rate-limit cooldown 15 min, consecutive-failure counting.
- **GeminiTier** uses the native generateContent API with functionDeclarations and systemInstruction; handles 429, 503, 400 and 404 (retired model).
- **OpenAiCompatibleTier** speaks standard chat completions with native tool_calls; handles 429 and 401/403; tool results as role: tool.
- **MockAiService** and **LocalTemplateEngine** provide empathetic rule-based offline replies (greetings, explanations, emotions, thanks, memory).

Routing, prompts and tokens

- **DeterministicLayer**: local pattern matching for quick-add (0.95), calendar (0.9) and board (0.85); tryHandle() and match().
- **ConfidenceRouter**: modes INSTANT, SMART (default) and AI_ONLY; executes only when confidence ≥ 0.9 (default) and the action is reversible; undo supported.
- **PromptBuilder**: persona, time/date/timezone/locale, user name and concerns, up to 6 memories and 4 capabilities, voice rules (spoken aloud, no markdown, 2-3 sentences), non-native tool-call format; fitHistory() trims to 8 turns / 6000 chars.

- **PromptAssembler**: single assembly path (system + history + pinned + task brief); re-assembles for failover providers with a new token budget; trims oldest first.
- **TokenEstimator**: English 4.0, Devanagari 2.2, other 3.0 chars per token; fitWithinBudget() keeps the most recent messages.
- **PromptedToolParser**: tolerant parser for fenced code-block tool calls from models without native tools (smart quotes, trailing commas).



Capabilities, agent modes, safety and memory

Capability executor	Actions
TaskCapability	create_task, list_tasks, complete_task, delete_task (destructive)
CalendarCapability	create_event, read_calendar, delete_event (destructive), find_free_slots
BoardCapability	add_board_item, list_board_items, delete_board_item (destructive)
MemoryCapability	save_memory, list_memories, delete_memory (destructive)
SettingsCapability	update settings
NavigationCapability	navigate screens
MediaCapability	media playback control
DateTimeCapability	date and time queries
GymCapability	log exercise sets, create exercises

Each Capability declares id, description, risk (READ / WRITE / DESTRUCTIVE) and preconditions. CapabilityRouter resolves CapabilityRequest (id, args, idempotency UUID) to a CapabilityResult: Done (with undo data), RequiresApproval, NotReady or Failed.

Agent modes

Mode	Behaviour	Steps
Instant	Deterministic only	1
Smart	Deterministic first, then AI	6
AI Only	Skip deterministic	10
Orchestrator	Subtasks (max depth 2, max 6 subtasks)	12
Plan	Read-only, denies writes	8
Explore	Read-only, denies all writes	6

Personality modes (SnehaMode): Companion, Planner, Executor, Study. PermissionRuleset has allow/ask/deny sets with deny-wins; AutoApprovePolicy limits consecutive auto-approvals (default 3); Checkpoints snapshot pre-images before WRITE/DESTRUCTIVE tools for undo; AgentOrchestrator enforces approval gates and returns AgentStepResult.

Safety

- **SchemaValidator / ToolSchema**: JSON Schema 2020-12 (type, required, enum, min/max, length, pattern); BoundedSelfRepair with max 2 retries.
- **PromptInjectionGuard**: wraps tool output, MCP responses, PDF text, web pages and nearby chat in <<<UNTRUSTED_DATA_BEGIN/END>>> delimiters with SHA-256 hash; source tag sanitised.
- **McpClient** (spec-compliant): initialize handshake, session handling, tools/list, tools/call, listChanged, SSE fallback, timeouts, cancellation. A legacy HTTP GET client remains and is being replaced.

Memory system

- **MemoryExtractor**: rule-based detection of facts, preferences and goals ("my name is", "i like", "i want", "i plan to", "remember", "don't forget", etc.).
- **MemoryRetriever**: BM25-style keyword scoring with stop-word filtering and term-frequency weighting; returns top-N relevant memories.
- **MemoryRepository**: CRUD with dedup, pinning, sensitivity flags and importance levels.

SECTION 17 · TEMPLATE SLIDE: Reference

Data layer and security

Database

SnehaDatabase (Room v16) holds 22 entities: Message, Memory, Task, TaskProject, TaskSubtask, Calendar, CalendarEvent, Board, BoardItem, BoardItemChecklist, ActionLog, ConversationSummary, TaskRun, Checkpoint, Alarm, AlarmEvent, Exercise, WorkoutSession, SetLog, Habit, HabitCompletion, JournalEntry. DAOs cover each area plus HubSearchDao.

An FTS4 table **global_search_index** (sourceType, sourceId, title, body) is kept in sync by insert/update/delete triggers across tasks, calendar events, board items, memories, messages, exercises, habits and journal entries. Sixteen migrations (1 to 16) cover schema history.

Repositories

Repository	Responsibility
ChatRepository	Message persistence, AI delegation, auto-memory extraction, streaming, tool-loop helpers
TaskRepository	CRUD, natural-language quick-add, projects, recurrence
CalendarRepository	Event CRUD, day arithmetic, grid math, overlap detection
BoardRepository	Board and item CRUD, six-colour sticky palette, grid step
MemoryRepository	CRUD, dedup, pinning, sensitivity, importance
HabitRepository	CRUD, daily toggle, archive
JournalRepository	Entries with mood tracking
GymRepository	5 default exercises, sessions, set logging, Epley 1RM PR detection, progression
HubSearchRepository	FTS4 global search

Security architecture

Component	Design
SecureKeyStoreManager	AES-256 GCM via hardware-backed AndroidKeyStore; user authentication (biometric / PIN); refuses plaintext storage on KeyStore failure
KeyDerivation	PBKDF2-HMAC-SHA256, 600k iterations, 32-byte salt, AES-256 GCM - fallback if hardware KeyStore is unavailable
SecureDbProvider	SQLCipher SupportFactory; 32 random-byte passphrase encrypted under the hardware key; opt-in via secureTables; decryption failure marks DB unreadable
PreferencesManager	DataStore keys incl. onboarding, theme, check-in times, provider and tier order, private mode, instant commands, custom endpoint; API keys stored through SecureKeyStoreManager; legacy migration at startup
DataWipeManager	Data wipe via Privacy centre
Threat model	Documented in docs/THREAT_MODEL.md

SECTION 18 · TEMPLATE SLIDE: Reference

Voice, alarms and companion services

Voice system

- **VoiceManager**: Android offline STT/TTS; female voice selection, pitch and rate control; strips markdown/emoji; speak(text, onDone); startListening(onResult, onError).
- **SnehaVoiceService** (foreground, microphone): wake word, "Yes?", listen, AI/tool loop, speak, re-arm. Proactive morning/evening check-ins. Checks RECORD_AUDIO before starting.
- **WakeWordEngine**: sherpa-onnx KeywordSpotter (reflection-based, optional) using the zipformer-gigaspeech 3.3M model; falls back to SpeechRecognizer transcript parsing.
- **VoiceCommandText**: wake phrases in English, Hindi, Spanish, French ("sneha", "hey sneha", "oye sneha", "hola sneha", "bonjour sneha" and Hindi variants) with inline commands.
- **AudioRecordBridge**: AudioRecord at 16 kHz mono PCM, 10 ms chunks for streaming KWS.

Alarms and companion

Component	Role
AlarmScheduler / AlarmClockScheduler	setExactAndAllowWhileIdle scheduling, canScheduleExactAlarms, snooze
AlarmScheduleCalculator / AlarmStateMachine	Next-alarm computation and state transitions
AlarmLinkingEngine	Links alarms to calendar events or gym sessions with an offset
AlarmRingingActivity + services	Full-screen ringing UI (showWhenLocked, turnScreenOn), foreground playback
AlarmReceiver	Re-registers after boot, package replace, time or timezone change; check-in morning/evening
Quick Settings tile and widget	AlarmTileService, AlarmQuickSettingsTileService, AlarmAppWidgetProvider
CheckInScheduler / CheckInReceiver	Alarm-based morning/evening reminders (not resident)
WebPlaybackService	Foreground media service for in-app web player
BraveLauncher	Opens URLs in Brave by package
MorningBriefing	Daily briefing generation

Gym and task engines

- **EpleyCalculator**: one-rep-max estimation; **ProgressionEngine**: next weight/rep recommendations from history.
- **QuickAddParser**: natural-language task parsing (title, due date, priority, tags, recurrence, project).

SECTION 19 · TEMPLATE SLIDE: Reference

Nearby chat and PDF

Nearby chat (BitChat integration)

- Opt-in Bluetooth mesh built on :bitchat and :bitchat-noise.
- **Nearby room**: public messages visible to mesh participants - not end-to-end encrypted.
- **Direct messages**: one-to-one over BitChat's Noise-encrypted BLE session; peer identities are not independently verified in the initial interface.
- Discovery stops when the screen is left or the app is backgrounded. Reached from Home using existing glass surfaces (no new bottom-nav tab).

PDF system

- **PdfEngine** renders pages with PdfRenderer; text extraction currently returns an empty string (Phase 0 - PdfBox not yet integrated).
- **PdfFileLibrary**, **PdfJob** (background processing), **PdfViewerActivity** (exported, handles VIEW for the PDF MIME type), **PdfViewerContent** (Compose viewer).
- PDFs shared from the system share sheet are copied to cache and opened in the viewer; extracted PDF text is wrapped by the injection guard.

Performance

TokenCache; per-script token budgeting; single shared video decoder behind the NavHost rather than one per screen.

SECTION 20 · TEMPLATE SLIDE: Reference

UI, navigation and theming

Navigation routes

onboarding · home · chat · nearby_chat · tasks · calendar · board · memories · settings · task_history · alarms · gym · habits · journal · focus · music · youtube · pdf_viewer · pdf · search

Screen packages

home, chat, board, calendar, tasks, memory, habits, journal, focus, gym, alarm, settings, media, navigation, onboarding, theme, components, nearby, pdf, privacy, agent.

Theme system

- Eight palettes (Lavender, Rose, Ocean, Mint, Amber, Slate and two more) via glassColors().
- ThemeMode system / light / dark; accessible TextSize scaling; Android 12+ Material You dynamic colour.
- Glassmorphic translucent surfaces with dynamic lighting and looping background video.
- Components: GlassSurface, GlassHeader, GlassListRow, ScreenScrim, LoopingVideoBackground, SharedVideoPlayer, SiteWebView, SnehaAvatar, SplashVideoOverlay, AppGlassBackground; NavTransitions for tab/push/pop.

SECTION 21 · TEMPLATE SLIDE: Reference

Android manifest summary

Item	Contents
Permissions	INTERNET, RECORD_AUDIO, POST_NOTIFICATIONS, FOREGROUND_SERVICE (+MICROPHONE, +MEDIA_PLAYBACK), RECEIVE_BOOT_COMPLETED, SCHEDULE_EXACT_ALARM, USE_FULL_SCREEN_INTENT, BLUETOOTH and BLUETOOTH_ADMIN (maxSdk 30), ACCESS_FINE_LOCATION (maxSdk 30), BLUETOOTH_SCAN (neverForLocation), BLUETOOTH_CONNECT, BLUETOOTH_ADVERTISE
Services	SnehaVoiceService (microphone), WebPlaybackService (mediaPlayback), AlarmRingingService / AlarmRingingForegroundService (mediaPlayback), AlarmTileService
Receivers	AlarmReceiver, CheckInReceiver, AlarmAppWidgetProvider
Activities	MainActivity (launcher, singleTask, deep links), AlarmRingingActivity (showWhenLocked, turnScreenOn), PdfViewerActivity

SECTION 22 · TEMPLATE SLIDE: Reference

Key design decisions

#	Decision	Rationale
1	Manual DI (Hilt introduced gradually for new code)	Keep the codebase simple while migrating safely
2	No plaintext fallback for secrets	KeyStore failure means refuse, never silently weaken security
3	Deterministic-first	Speed and offline operation
4	Confidence-gated execution	Auto-run only at ≥ 0.9 confidence and when reversible
5	Deny-wins permissions	Per-mode rules can narrow but never widen the global deny set
6	Prompt-injection defence	All untrusted content delimited and hashed
7	Circuit breaker	Failing providers benched; rate-limited ones for 15 minutes
8	Per-script token estimation	Devanagari-aware budgeting
9	SQLCipher opt-in	Encrypted DB only when the user enables secureTables
10	Voice as foreground service	Microphone access requires it on Android 14+
11	Single activity, singleTask	NavHost with all routes
12	Shared video background	One decoder behind NavHost, not per screen

Documentation set: ARCHITECTURE.md, AUDIT.md (Phase 0 gaps and licence triage), CHANGELOG.md, THIRD_PARTY.md, STUDY_NOTES (licence triage; agent modes from Kilo Code; tool loop from Cline; MCP Kotlin SDK), docs/THREAT_MODEL.md, and 17 studied third-party repositories (git-ignored).

SECTION 23 · TEMPLATE SLIDE: Supports slides 9, 12, 13

Risks and mitigations

Derived from the documented limitations **[PROPOSED]**.

Risk	Impact	Mitigation
Free-tier quotas or terms change	Provider disappears or throttles	10-provider cascade, circuit breaker, custom endpoint, local Ollama, deterministic fallback
Prompt injection via PDFs / web / MCP / nearby chat	Unintended tool actions	Delimiter and hash wrapping, approval for destructive tools, schema validation, deny-wins permissions
Public nearby room is unencrypted	Users assume privacy	Clear UI labelling; direct messages use Noise; add peer verification
Key or DB theft from a compromised device	Credential or data exposure	Hardware-backed keys, optional SQLCipher, no plaintext fallback, data wipe
Dependence on reflection-based optional wake-word library	Wake word unavailable on some devices	Transcript-based wake-phrase fallback
Battery and background restrictions	Missed alarms or voice service killed	Exact alarms with boot re-registration, foreground service, check-ins alarm-based (not resident)
GPLv3 limits closed-source monetisation	Narrower revenue options	Service-based model (managed cascade, sync, support, institutional)
Solo / small-team capacity	Slow delivery	Mentorship and infrastructure support from AICF; community contributions

SECTION 24 · TEMPLATE SLIDE: Checklist

Information still needed to complete the deck

These are the only items the project data could not provide. Fill them in and the 15-slide template can be completed directly from this report.

Slide	Missing item	Status
1-2	Team / startup name, presenter, date (DD/MM/YYYY)	☐
5	Cited TAM / SAM / SOM figures	☐
6	Screenshots and demo video; confirm development stage	☐
7	Final pricing and revenue choice	☐
8	Budget, funding ask, use of funds	☐
9	Verified competitor feature comparison	☐
11	Founder bios, roles, mentors	☐
12	Beta users, feedback, awards, grants	☐
15	Contact name, email / phone, QR or repo link	☐